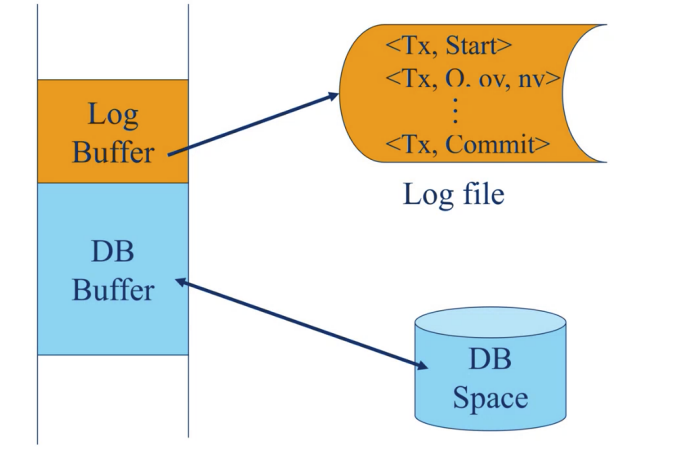
Recovery and Buffer Management (Log based recovery)

Quick Review

- Last time before a midterm, we have **log based recovery topic.**



  - idea is quite simple, we keep log record.
    - When we start transaction, we have <Tx, Start>
    - When the transaction is active during processing of transaction, we have <Tx, Q, ov, nv> (old value, new value)
    - When the transaction is committed, we have <Tx, Commit> output to the log file.
      - So the physical commit(when we have log-based recovery) → when commit record output to the log file successfully.
  - Based on this principle, During recovery time after failure, the system simply search commit record from the back

    - if the commit record is found → that transaction commit
    - otherwise → the transaction failed
- Even though, you have commit record is log buffer → it not yet commit (At failure time, the log record in log buffer maybe lose) → we count only commit in log file or commit in AIJ

> 👉 Old value/ new value
>
> - we use new value to redo → solve first problem (commit before output)
> - we used old value to undo DB space → solve second problem (output before commit)

Old value is store in BIJ, new value stored in AIJ → can choose to install one of them or both of them

---

Content

🌱 2 Approaches of log-based recovery

(Book has slight different terminology, same idea but different terminology)

Log-Based Recovery

- A **log** is a sequence of **log records**. The records keep information about update activities on the database.
  - The **log** is kept on stable storage
- When transaction T_i starts, it registers itself by writing a

  $<T_i$ **start**$>$ log record
- *Before* T_i executes **write**(X), a log record

  $<T_i, X, V_1, V_2>$

  is written, where V_1 is the value of X before the write (the **old value**), and V_2 is the value to be written to X (the **new value**).
- When T_i finishes it last statement, the log record $<T_i$ **commi**t$>$ is written.
- Two approaches using logs
  - Immediate database modification
  - Deferred database modification.

From book

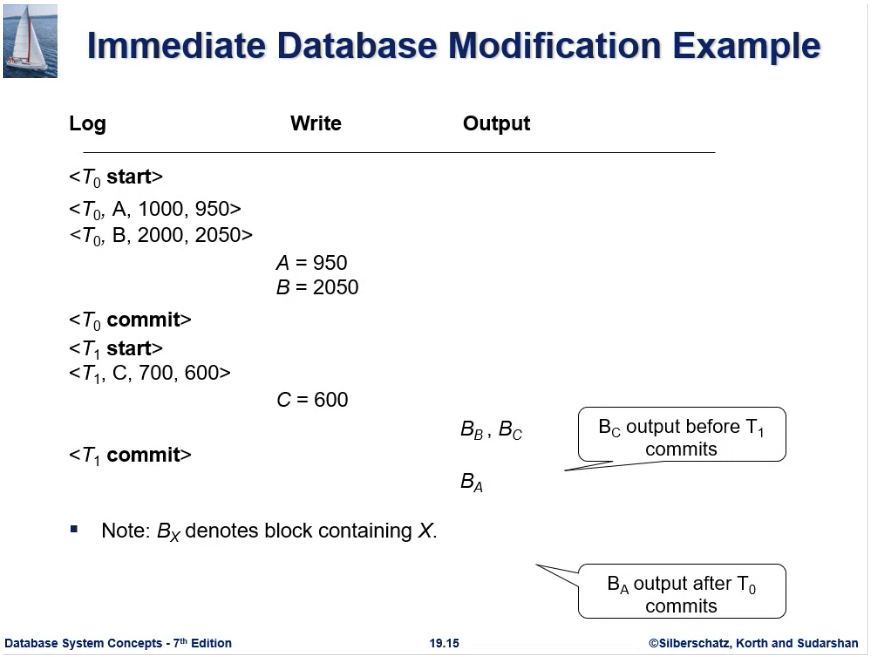
- There 2 approaches using log (log based recovery)

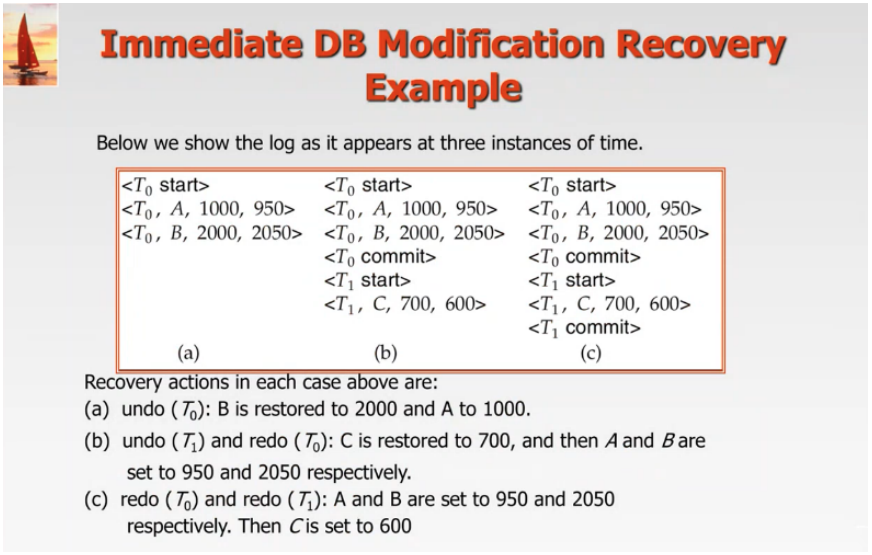
1. **Immediate database modification**
    - The immediate-modification scheme allows update of an uncommitted transaction to be made to the buffer, or the disk itself, before the transaction commits
    - Output may take place before or after commit

    👉 - in this categories
        - BIJ only is an immediate DB modification (take place before commit)
            - no new value → must output before commit
        - Both AIJ & BIJ is an immediate DB modification (output anytime)

Immediate Database Modification

- The **immediate-modification** scheme allows updates of an uncommitted transaction to be made to the buffer, or the disk itself, before the transaction commits
- Update log record must be written *before* database item is written
  - We assume that the log record is output directly to stable storage
  - (Will see later that how to postpone log record output to some extent)
- Output of updated blocks to disk can take place at any time before or after transaction commit
- Order in which blocks are output can be different from the order in which they are written.
- The **deferred-modification** scheme performs updates to buffer/disk only at the time of transaction commit
  - Simplifies some aspects of recovery
  - But has overhead of storing local copy



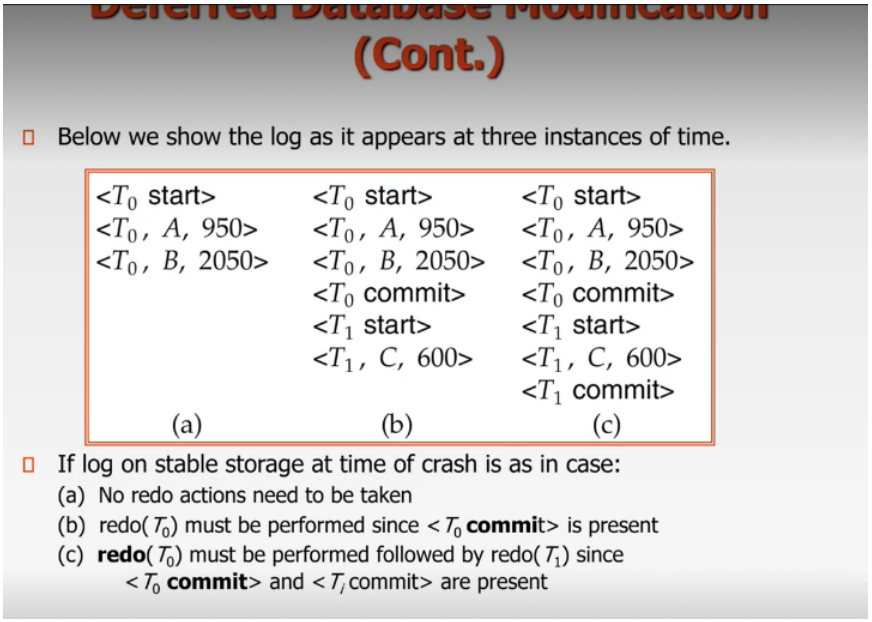


- in immediate database modification, we need an old value, because immediate imply that output can be done before commit, so we need old value for recovery process (undo)
- this diagram show both old and new value

2. **Deferred database modification (can be postpone)**
    - The deferred-modification scheme performs updates to buffer/disk only at the time of transaction commit
    - It must commit first - commit then later output

- Output may take place after commit or at the commit time

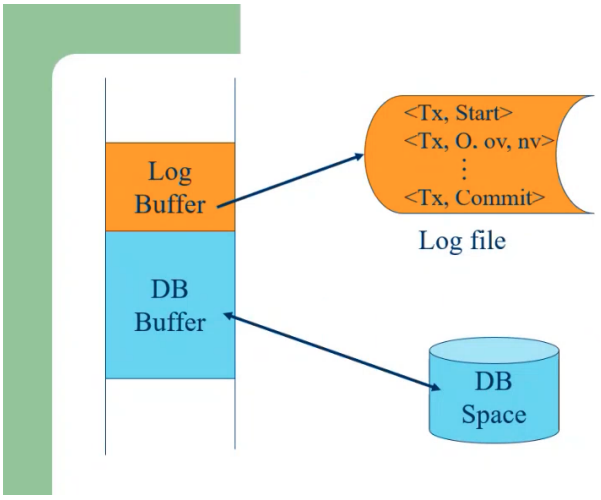
> 👉 AIJ only is deferred DB modification (we dont have old value, not allow db buffer to output to db space before commit)



- in this, it has only AIJ (new value)
- Transaction T0 transfer 50 dollars from Transaction A to B
  - Before the transaction A was 1000, B was 2000
  - After transaction A become 950, and B become 2050
  - The log only show new value (AIJ)
- For transaction T1, we withdraw 100 dollars from C (C was 700)
  - After withdrawal → become 600

Show only new value

🌱🌱 Buffer Management (19.5 - book)



from, this we see log buffer, db buffer and know that they are in main memory

- Log buffer and DB buffer are in main memory
- **Who manage the buffer (DB buffer and log buffer)?** Should it be manage by the DBMS or the OS?
  - when we study OS → we know that OS controls the main memory. It can paging, segmentation, ..., It also has page replacement strategy like FIFO, read recently used, read frequently used, ... that adopt by the OS
  - OS actually control the main memory, but should we let OS handle buffer management of this log buffer and database buffer?
  - (There were discussion and also implementation.) How much involvement should DBMS be in this buffer management activity?
    - after long discussion and several implementation, it is very clear that it is the DBMS that should be in charge of this buffer management.

> 👉 There are protocol that the DBMS must execute and must guide, force DBMS to follow **such protocol.**

Those Protocol?

(Some topic from book)

1. **Log-Record Buffering** - Log record are buffered in main memory, instead of being output directly to stable storage and it need to be managed.

Log Record Buffering

- **Log record buffering**: log records are buffered in main memory, instead of being output directly to stable storage.
  - Log records are output to stable storage when a block of log records in the buffer is full, or a **log force** operation is executed.
- Log force is performed to commit a transaction by forcing all its log records (including the commit record) to stable storage.
- Several log records can thus be output using a single output operation, reducing the I/O cost.

- Log force is performed to commit a transaction by forcing all its log records (including the commit record) to stable storage.
  - This book always assume that log file is in stable storage → so it mean force to log file. (when we say force to stable storage)
- Several log records can thus be output using a single output operation, reducing the I/O cost.

👉 There an important protocol on **Log Record Buffering**

Log Record Buffering (Cont.)

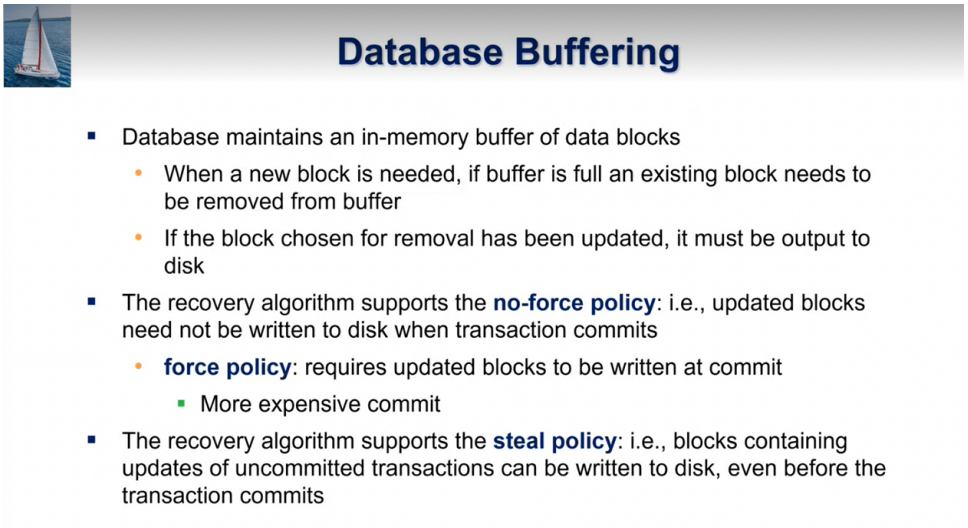
- The rules below must be followed if log records are buffered:
  - Log records are output to stable storage in the order in which they are created.
  - Transaction T_i enters the commit state only when the log record <T_i **commit**> has been output to stable storage.
  - Before a block of data in main memory is output to the database, all log records pertaining to data in that block must have been output to stable storage.
    - This rule is called the **write-ahead logging** or **WAL** rule
    - Strictly speaking, WAL only requires undo information to be output

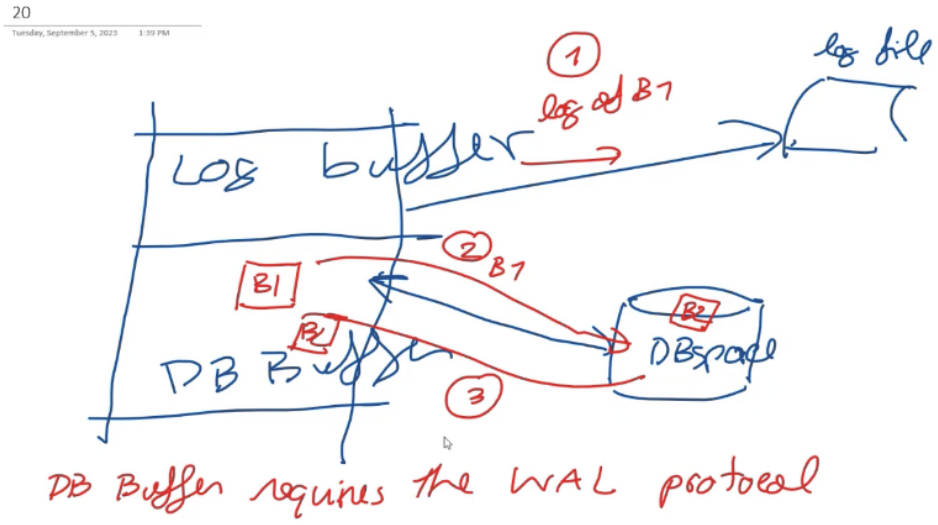
Rule of Log-record buffering

- Log records are output to stable storage in the order in which they are created
  - this imply that commit record must be the last one of that transaction.
- Transaction Ti enters the commit state only when the log record <Ti commit> as been output to stable storage. (to ensure physical commit)
- WAL → before block of data output to the database, log record output to stable storage first.

> 👉 The DBMS must follow this rule
>
> - For OS point of view, DBMS is just an application program for OS
>   - So, this is application requirement of DBMS for log based recovery.
> - It must be DBMS that instruct the OS that ok, you must go now or wait → they must be test together.

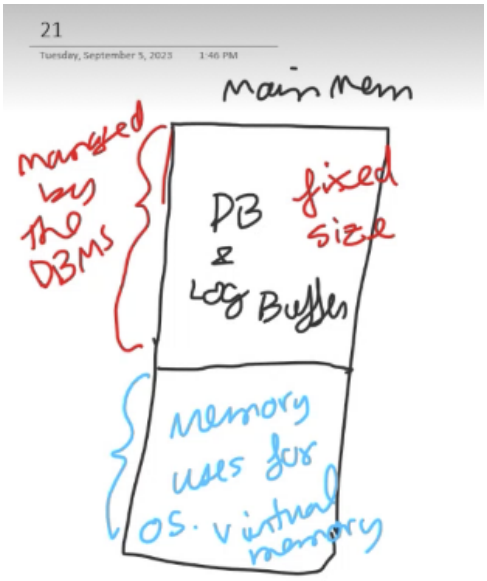
2. **Database Buffering**





- Suppose you have block b1 in DB buffer and a block b2 in DB space → in practice, after a while your database buffer gonna be full because you use data (input data to buffer), and if you want b2 to be input in DB buffer, you can't just input b2 directly to buffer → b2 has to replace a block here (DB space is full). → bring b1 out before you move b2 in
- Step (database buffer simple protocol)
  - When you bring b1 out, you need to output the log of b1 first to the log file
  - b1 back to DB space
  - b2 in to db buffer
- DB buffer also requires the WAL protocol

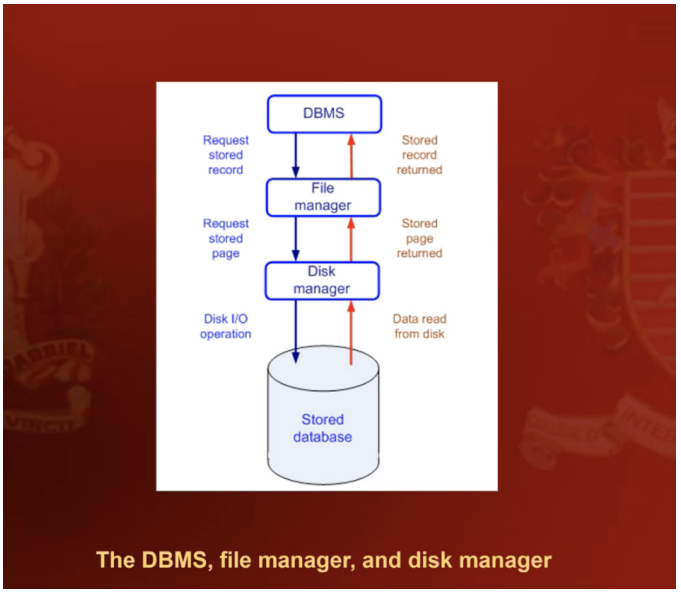
3. **Operating System role in buffer management**
   - We can manage the database buffer by using one of two approaches:
   1. The database system reserves part of main memory to serve as a buffer that it(DBMS), rather than the operating system, manages.
      - DBMS reserves part of main memory to serve as a buffer that the DBMS itself manages



- From main memory pov, the os allow the DBMS to reserves part of the main memory as database and log buffer
- size of database and log buffer is fixed size → The DBMS occupied a fixed size of main memory and it's managed by the DBMS.
- It's the DBMS that manage the protocol without passing through the OS.
- blue part → virtual memory

2. The database system implements its buffer within the virtual memory provided by the operating system.
    - it is the DBMS that implement it buffer within the virtual memory provided by the OS.

Diagram show interaction between DBMS and OS.



The DBMS, file manager, and disk manager

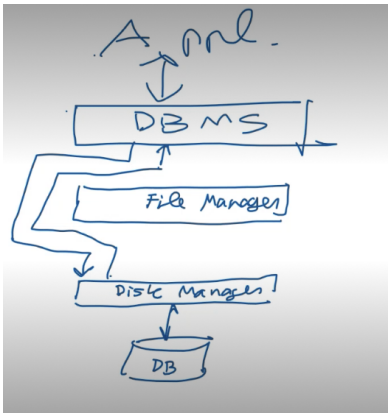
- the dbms interact with the file manager of the OS, and the file manager interact with the disk manager of the OS, then retrieve data, then the file manager read the data in physical block or OS block.
- The dbms read the data in logical block or database block (this is conventional one)

(3.1)Configuration 1 → database system reserves part of main memory to serve as a buffer.....

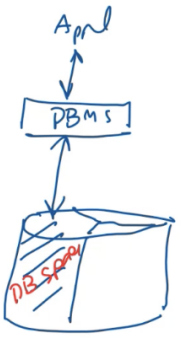
(buffer fixed, DBMS talk directly to disk manager)

(from the implementation point of view, you don't see file → you see divide) → we normally called this the raw device option

- But our configuration number 1 → the DBMS simple bypass the file manager, and talk directly to the disk manager.



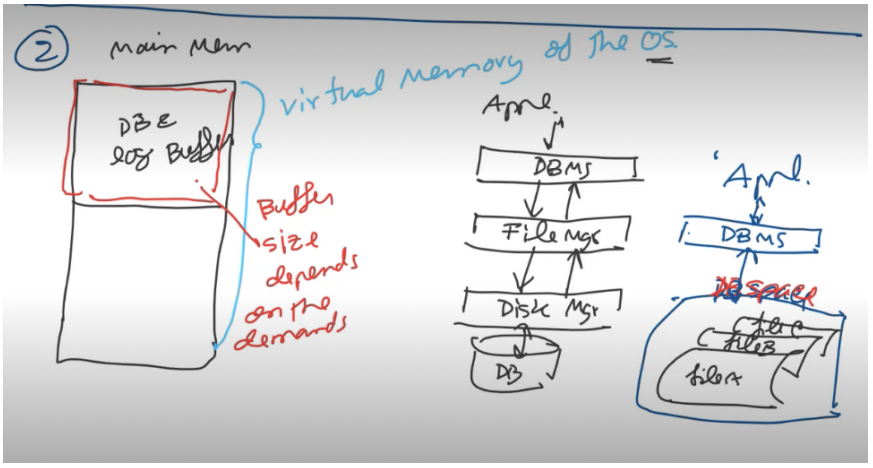
- and the disk manager return the block that bypass the file manager. It's the DBMS that manage the main memory.



- from the storage point of view, this is the physical disk, you may partition it into some partition (logical device) → and you allocate this divice to be DB space, so it's the DBMS who interact directly to db space.

(3.2)Configuration 2 → The database system implements its buffer within the virtual memory provided by the operating system

- the virtual memory cover the main memory and extended to the disk, ofcourse.
- the database buffer and log buffer is inside this (virtual memory).
- this one is also control by the OS.
- the size of the buffer is not fixed → the size depends on the actual demand, which mean that if you have more user, the system enlarge it. if you have less user, the system reduce the size of database buffer.



more popular configuration compare to number 1, number 1 used to be more popular in the past.

- from the storage point of view, you may have several file, you have to prepared those file first, and combine those file together into the DBspace.
- Nowadays, hardware is quit good, main memory is quite cheap → you can have buffer in virtual memory, and it can be extended, enlarge on demand
  - when you have less load → the system reduce the size of the buffer → so you have room for other application
  - but when you have more database load → the size of buffer increase to accommodate more data

The popularity of 2 configuration and why?

- in the past the first is more popular, but now the second is more popular.
- in the past, the hardware was not very good and the OS and the DBMS may not talk to each other very well (they were not tested together)
  - in the case you use DBMS that have not been test with the particular OS → it better to use first configuration so the DBMS will exercise the protocol (it the DBMS that manage data transfer between buffer and the disk) the DBMS control all the transfer based on the WAL. (so this was the better choice in the past, because the OS may not be test with the DBMS)
    - for open source dbms, this is also a popular choice → let's the dbms manage it.
    - However this number 1, it's the admin (DBA) who has to estimate the size of database buffer (because it fixed) → estimate the best size

for your organization

Some case study

- first, we use the default size → increase buffer size → get better response time from database application. (because the size of buffer is relatively big → have less disk i/O (and i/o take time)
- try again → enlarge the size of buffer to something like 70% → performance drop → because we didn't had enough room for other process to run (most of main memory is for the buffer. (so, we have something like 25% ? for other process including DBMS to run → not good.
  - many process need to be swap in and out and use only 25%.
  - enlarge too much → too small virtual memory (if you use configuration number 1 → use proper size, but have to do it manually)

- configuration 1 is also good if you want your best performance form hardware because you bypass file manager (some file manager is slow)
- from some experiment, by bypass file manger → it will be 20% better.

- but with good hardware, and the dbms and OS test together → number 2 is common choice

---

How long should log records be buffer?

(2 kind of implementation)

1. Not long, log records should be output to log files as soon as possible, log records are forced output to log files after each commit.
   - this choice is obvious, because if you keep log record long time in buffer, and failure take place → you lost log record, include commit record → so you may lost many transaction.
   - this approach → when it see commit record → force output the log
   - the idea → to minimize the lost when having failure.
2. Buffer the log records until a log buffer block is full.
   - the idea → to reduce the disk i/o.

- group commit approach
- nowadays, most DBMS use the group commit approach to reduce disk i/o acticity and space
- make sure the buffer block is full → then go in 1 block
- popular → nowadays the server won’t go down very often.
